

# TradeCore — Python FastAPI & Clean Architecture Kılavuzu

Bu kılavuz, TradeCore projesini kurumsal .NET mimarisinden yerel-öncelikli (Local-First), sıfır maliyetli ve yüksek veri işleme kabiliyetine sahip Python FastAPI + SQLite mimarisine dönüştürme şartnamesidir.

## 1. Proje Vizyonu ve Mimari İlkeler

- **Hedef:** Kişisel finans yönetimi, BIST hisse/portföy takibi, banka ekstresi (PDF/Excel) ayrıştırma, RSS haber analizi ve yapay zeka (Gemini) destekli finansal raporlama platformu.
- **Local-First & Sıfır Maliyet:** Tüm veriler yerel diskte tek bir SQLite (`tradecore.db`) dosyasında saklanır. Bulut veritabanı masrafı yoktur.
- **Next.js 15 Frontend:** Mevcut modern Next.js 15 App Router ve Tailwind/shadcn arayüzü korunur; Python API'ye bağlanır.
- **Telegram Bot Entegrasyonu:** Mobil cihazdan tek mesajla harcama veya portföy işlemi kaydetme desteği.

## 2. Cursor İçin Mimari Kurallar (.cursorrules)

1. **Modüler Katman Ayrımı:**
  - `core/`: Güvenlik, JWT token üretimi, ayarlar (`config.py`).
  - `models/`: SQLAlchemy / Pydantic veritabanı entity'leri.
  - `services/`: İş mantığı (Ekstre ayrıştırma, yfinance piyasa verisi, Gemini AI raporlayıcı, Telegram botu).
  - `api/`: FastAPI route controller'ları.
2. **Tip Güvenliği:** Tüm Python fonksiyonları Pydantic şemaları ve type hinting (`str`, `int`, `Optional[T]`) ile yazılmalıdır.
3. **API Uyumluluğu:** Next.js frontend'in mevcut auth akışını (`POST /api/auth/login`, `POST /api/auth/register`, `GET /api/auth/me`) bozmamak için auth uçları aynı JSON ve HttpOnly çerez yapısıyla dönmelidir.

### 3. Yeni Klasör Yapısı

TradeCore/

```
├── .cursorrules          # Yeni Python mimari kuralları
├── docker-compose.yml    # FastAPI + Next.js + SQLite volume
├── frontend/            # Mevcut Next.js 15 App Router arayüzü
│   ├── app/
│   │   ├── (auth)/
│   │   └── (dashboard)/
│   └── lib/api.ts
└── backend/
    ├── requirements.txt  # FastAPI, SQLAlchemy, yfinance, pdfplumber vb.
    ├── main.py           # FastAPI ana giriş noktası, CORS, middleware
    ├── app/
    │   ├── core/
    │   │   ├── config.py # Ortam değişkenleri (.env)
    │   │   └── security.py # BCrypt şifreleme ve JWT üretimi
    │   └── db/
    │       ├── session.py # SQLite bağlantısı ve session yönetimi
    │       └── models/
    │           ├── user.py # User & RefreshToken modelleri
```

```
| | └─ portfolio.py      # Holding (Hisse) ve Transaction modelleri
| | └─ statement.py     # BankStatement ve ParsedRecord modelleri
| | └─ ai_report.py     # AiReport ve MarketNews modelleri
| └─ services/
| | └─ market_service.py # yfinance BIST & altın/döviz çekici
| | └─ parser_service.py # pdfplumber PDF/Excel banka ekstre
okuyucu
| | └─ ai_service.py     # Gemini API portföy analiz motoru
| | └─ telegram_bot.py   # python-telegram-bot arka plan dinleyicisi
| └─ api/
|   └─ auth.py          # Register, Login, Refresh, Me uçları
|   └─ portfolio.py     # Hisse ve varlık yönetimi uçları
|   └─ parser.py        # Drag-and-drop ekstre yükleme uçları
|   └─ analysis.py      # Yapay zeka analiz raporu üretim uçları
```

## 4. 5 Aşamalı Uygulama Planı

### Aşama 1: Python FastAPI Temeli & Mevcut Auth Yapısının Korunması

- `backend/requirements.txt` dosyasının oluşturulması (`fastapi`, `uvicorn`, `sqlmodel`, `python-jose`, `passlib[bcrypt]`, `python-multipart`).
- `core/security.py` ile BCrypt şifreleme ve JWT Access/Refresh token mekanizmasının yazılması.

- Next.js frontend'in beklediği `POST /api/auth/register`, `POST /api/auth/login`, `GET /api/auth/me` uçlarının birebir aynı yanıt formatında tamamlanması.
- SQLite veritabanı bağlantısının (`tradecore.db`) kurulması.

## Aşama 2: Borsa & Piyasa Servisi (BIST, Altın, Döviz)

- `services/market_service.py` modülünün yazılması (`yfinance` kütüphanesi ile).
- Borsa İstanbul (THYAO.IS, ASELS.IS vb.), Gram Altın ve Dolar/Euro kurlarının anlık çekilmesi.
- `models/portfolio.py`: Kullanıcının sahip olduğu hisseleri (`Holding`) ve alış/satış geçmişini tutan modellerin eklenmesi.
- `GET /api/portfolio`: Kullanıcının anlık kâr/zarar ve portföy değerini dönen uç nokta.

## Aşama 3: Banka Ekstresi Ayırıştırıcı Motor (PDF & Excel)

- `services/parser_service.py`: `pdfplumber` ve `openpyxl` kullanarak banka hesap özetlerinden tarih, açıklama ve tutar kolonlarını ayırıştıran servis.
- Harcama açıklamalarından (Örn: MIGROS → Market, NETFLIX → Abonelik) otomatik kategori tahmini yapan kural motoru.
- `POST /api/parser/upload`: Next.js arayüzünden sürükle-bırak ile ekstre yükleme ucu.

## Aşama 4: Yapay Zeka Analiz Motoru (Gemini AI)

- `services/ai_service.py`: Portföydeki hisseleri, son piyasa verilerini ve harcama trendlerini yapılandırılmış prompt ile Gemini API'ye ileten servis.
- Portföy çeşitlendirmesi, risk skoru (1-10) ve hisse bazlı strateji bülteni üreten komut.
- `POST /api/analysis/generate` ve `GET /api/analysis/reports` uçları.

## Ařama 5: Telegram Botu & Next.js Borsa Dashboard'u

- `services/telegram_bot.py`: Arka planda alıřan asenkron Telegram botu (`/harca 120 kahve`, `/durum`, `/hisse`).
- Next.js 15 dashboard ekranının; Portföy Özeti, Hisse Takibi, Ekstre Yükleme ve Yapay Zeka Rapor panelleriyle zenginleřtirilmesi.